**SURE ProEd**

*Skill Up-Gradation for Rural Youth Empowerment*

# Smart Supply Chain & Inventory Management System

## Project Report

*A Full-Stack Web Application for Real-Time Inventory and Supply Chain Operations*

| | |
|---|---|
| **Student Name** | Settipalli Aruna |
| **Domain** | Full Stack Development |
| **Project Title** | Smart Supply Chain & Inventory Management System |
| **Technology Stack** | Java, Spring Boot, React.js, PostgreSQL |
| **Authentication** | JWT (JSON Web Token) |
| **API Architecture** | RESTful APIs |
| **Database** | PostgreSQL (Open Source RDBMS) |
| **Organization** | SURE ProEd – Skill Up-Gradation for Rural Youth Empowerment |
| **Submission Date** | 28 April 2026 |

# 1. Abstract

The Smart Supply Chain & Inventory Management Platform is a web-based application designed to streamline and optimize the management of products, inventory, and warehouse operations. In today's fast-paced business environment, efficient inventory tracking and supply chain coordination are critical for reducing operational costs and improving productivity.

This project provides a centralized system where administrators can manage products, monitor stock levels, and track inventory across multiple warehouses. The platform enables real-time updates of inventory quantities, ensuring that stock levels remain within defined minimum and maximum thresholds. By implementing role-based access and secure authentication, the system ensures data integrity and controlled access to sensitive operations.

The application is developed using modern backend technologies such as Java with Spring Boot and PostgreSQL for robust data handling, along with RESTful APIs for seamless communication between frontend and backend systems. The frontend, built with React.js, provides an intuitive user interface for easy navigation and efficient management of resources.

Key features of the system include product management, inventory tracking, warehouse management, and secure authentication using JWT (JSON Web Token). The system is scalable and can be extended to include advanced features such as analytics, demand forecasting, and automated stock alerts.

# 2. Introduction

In the modern digital era, efficient management of supply chains and inventory systems has become a critical requirement for organizations across industries. Businesses rely heavily on accurate tracking of products, real-time stock updates, and streamlined warehouse operations to ensure smooth functioning and customer satisfaction. Traditional inventory management methods, which often involve manual record-keeping or fragmented systems, are prone to errors, inefficiencies, and delays.

The Smart Supply Chain & Inventory Management Platform is developed to address these challenges by providing a centralized and automated system for managing products, inventory, and warehouse operations. The primary objective of this project is to design and implement a scalable and efficient system that enables administrators to monitor and control stock levels across multiple warehouses in real time.

This platform allows users to manage product details such as category, stock thresholds, and expiry dates, while also tracking inventory quantities and warehouse locations. By integrating these components into a unified system, the application ensures better visibility, improved decision-making, and enhanced operational efficiency.

The system is built using Java and Spring Boot for backend development, ensuring robust performance and secure API handling. PostgreSQL is used as the database for efficient data storage and retrieval. The frontend is built using React.js to provide a user-friendly interface enabling seamless interaction with the system.

One of the key highlights of this project is the implementation of RESTful APIs that enable smooth communication between the frontend and backend layers. Additionally, the use of JWT-based authentication ensures secure access control, allowing only authorized users to perform sensitive operations such as adding or updating products and inventory.

This project not only demonstrates the application of software engineering principles but also reflects real-world industry practices in building scalable backend systems. It serves as a strong foundation for further enhancements such as analytics, demand forecasting, automated alerts, and integration with external supply chain systems.

# 3. Scope of the Project

The scope of the Smart Supply Chain & Inventory Management Platform is centered around developing a scalable and efficient system to manage products, inventory, and warehouse operations in a centralized environment.

## 3.1 Core Functional Scope

The system enables administrators to add, update, and manage product details such as name, category, expiry date, and stock thresholds. It supports real-time tracking of inventory quantities across different warehouses, maintaining both minimum and maximum stock limits to prevent overstocking and shortages.

## 3.2 Architectural Uniqueness

A standout feature is the deliberate separation of product management and inventory management into distinct modules. Products are master data; inventory records the physical stock of those products per warehouse. This mirrors enterprise-level design, ensures scalability, and prevents data redundancy.

- Product Module – Manages the product catalogue, categories, pricing, and thresholds.
- Inventory Module – Manages real-time stock quantities per warehouse linked via foreign keys.
- Warehouse Module – Manages warehouse locations, managers, and capacity independently.

## 3.3 Security Scope

JWT (JSON Web Token) authentication ensures only authorized administrators can add products, update stock levels, and manage inventory, aligning with industry standards for API security.

## 3.4 Future Enhancement Scope

- Automated stock alerts when inventory reaches minimum threshold levels.
- Demand forecasting using machine learning and historical sales data.
- Multi-warehouse stock optimization and transfer management.
- Analytics dashboards for real-time business intelligence.
- Mobile application support through existing REST APIs.

# 4. Existing System vs. Proposed System

## 4.1 Existing System – Limitations

- Manual data entry increases the probability of human error.
- No real-time inventory updates; stock information becomes outdated quickly.
- Lack of integration between product, inventory, and warehouse data.
- Poor scalability; difficult to extend or grow with the business.
- Limited security controls and absence of proper access control.
- No support for multi-warehouse stock tracking.

## 4.2 Proposed System – Key Features Implemented

| Feature | Implementation | Benefit |
|---|---|---|
| Real-Time Inventory Tracking | Spring Boot REST APIs + PostgreSQL | Instant stock visibility across warehouses |
| Product Management Module | Full CRUD with JPA/Hibernate ORM | Centralized product catalogue management |
| Warehouse Management | Dedicated entity with manager & location | Clear organizational structure |
| JWT Authentication | Spring Security token-based auth | Secure access control for all endpoints |
| Stock Threshold Alerts | Min/max level tracking per product | Prevents stockouts and overstocking |
| RESTful API Architecture | GET, POST, PUT, DELETE endpoints | Flexible frontend-backend communication |
| Modular Architecture | Separated Product, Inventory, Warehouse | Easy to scale and extend |
| Bar Chart Dashboard | Chart.js + React-Chartjs-2 | Visual stock overview at a glance |

## 4.3 Comparison Summary

| Aspect | Existing System | Proposed System |
|---|---|---|
| Data Management | Fragmented manual spreadsheets | Centralized integrated database |
| Real-Time Updates | Not available | Full real-time inventory tracking |
| Security | Minimal or none | JWT-based authentication |

| | | |
|---|---|---|
| Architecture | Monolithic, hard to scale | Modular, microservice-ready |
| Multi-Warehouse | Not supported | Fully supported |
| Stock Alerts | Manual checking required | Threshold-based automated alerts |
| Scalability | Very limited | Highly scalable design |

# 5. Architectural Diagram

The system follows a three-tier architecture: Presentation Layer (React.js), Business Logic Layer (Spring Boot), and Data Layer (PostgreSQL). JWT security is applied as a cross-cutting concern across all API endpoints.

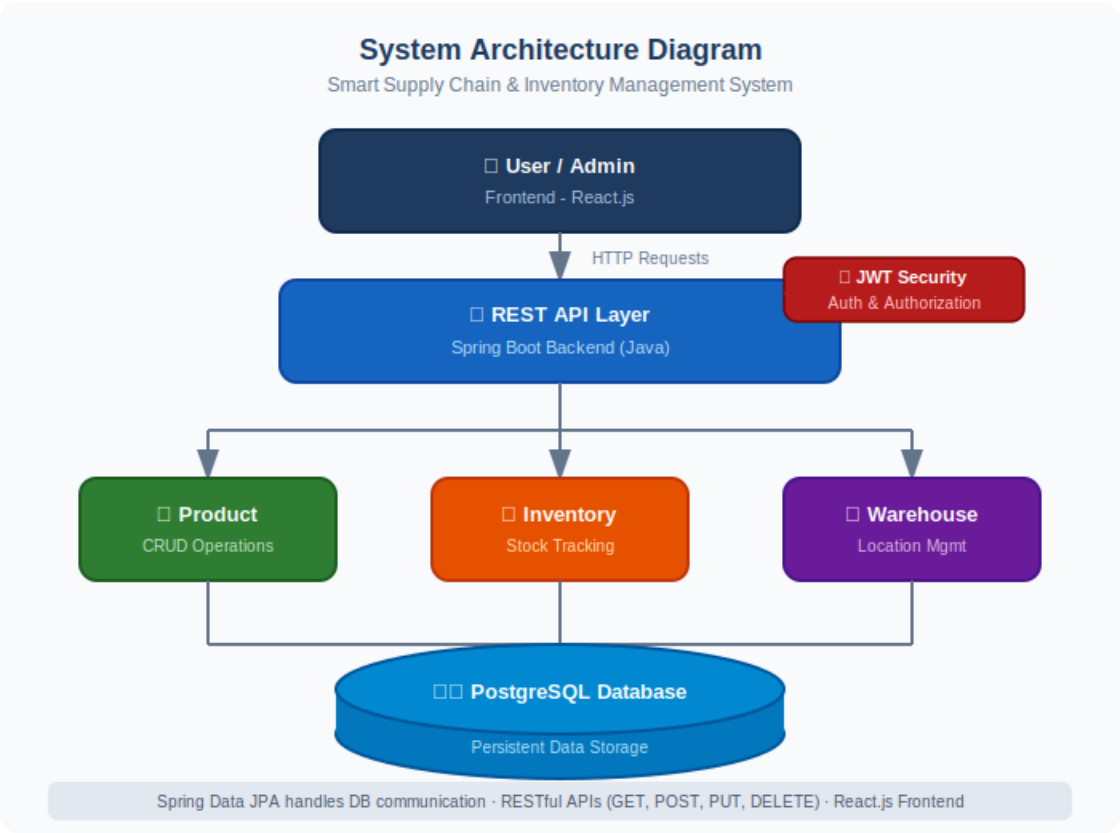


*Figure 1: System Architecture – Three-Tier Architecture with JWT Security*

## 5.1 Architecture Layers

### Presentation Layer — React.js Frontend

React.js provides a dynamic, component-based UI. Users interact through the browser; React manages state and renders real-time data from backend APIs. Chart.js integration (react-chartjs-2) powers the visual Stock Overview bar chart on the Inventory Dashboard.

### Business Logic Layer — Spring Boot Backend

Java with Spring Boot exposes RESTful endpoints. Three primary services — ProductController, InventoryController, and AuthController — handle all business logic. JWT middleware validates every request before processing.

### Data Layer — PostgreSQL Database

PostgreSQL stores all persistent data. Spring Data JPA with Hibernate ORM handles object-relational mapping. Three primary tables (product, inventory, warehouse) are linked via foreign keys enforcing referential integrity.

# 6. Entity Relationship Diagram (ERD)

The ERD illustrates three primary entities — Product, Warehouse, and Inventory — with Inventory acting as a junction table resolving the many-to-many relationship between products and warehouses.

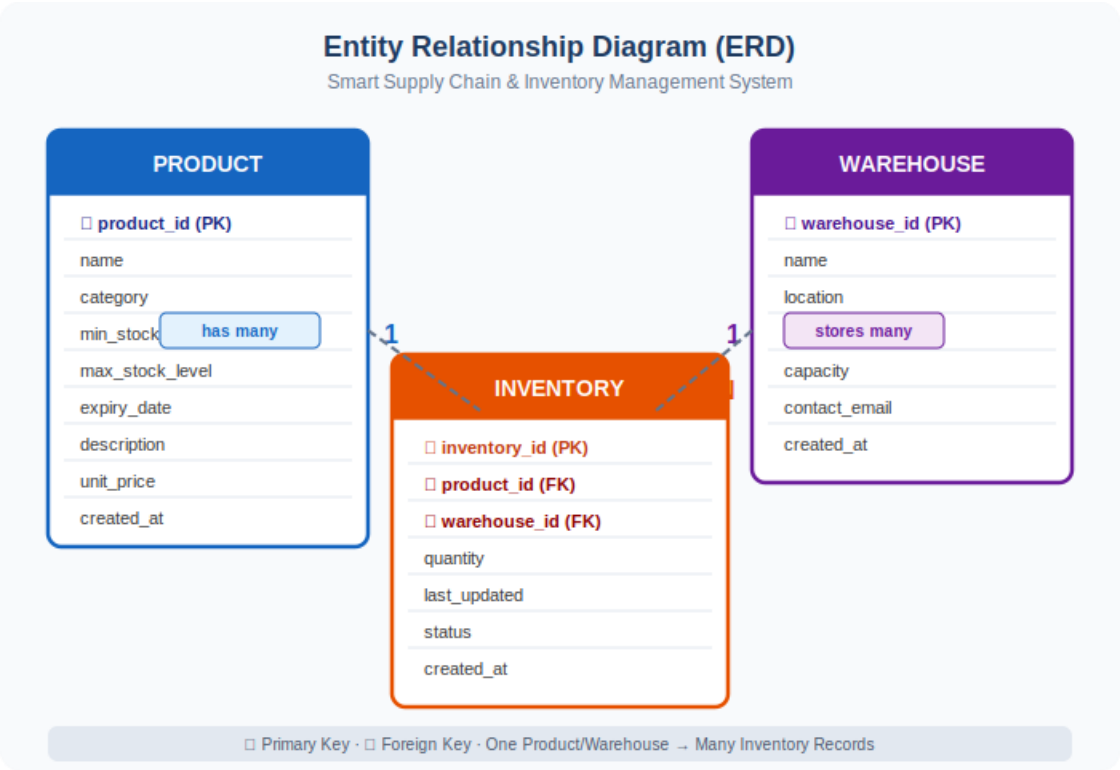


*Figure 2: Entity Relationship Diagram – Data Model*

| Relationship | Cardinality | Description |
|---|---|---|
| Product → Inventory | One-to-Many (1:N) | One product appears in many inventory records across multiple warehouses |
| Warehouse → Inventory | One-to-Many (1:N) | One warehouse holds inventory records for many products |
| Inventory (Junction) | Many-to-Many resolved | Bridges the Product-Warehouse many-to-many relationship |

# 7. Technology Stack

## 7.1 Frontend

| Technology | Purpose |
|---|---|
| React.js | Component-based UI, state management, dynamic rendering |
| HTML5 / CSS3 | Semantic structure, styling, and responsive design |
| JavaScript (ES6+) | Client-side logic, DOM manipulation, event handling |
| Chart.js + React-Chartjs-2 | Bar chart visualization for Stock Overview dashboard |
| Tailwind CSS | Utility-first CSS framework for styling (tailwind.config.js) |
| Vite | Fast build tool and development server (vite.config.js) |

## 7.2 Backend

| Technology | Purpose |
|---|---|
| Java | Core language for all backend business logic |
| Spring Boot | Rapid backend development, embedded Tomcat, auto-configuration |
| Spring Web MVC | RESTful API creation — @RestController, @RequestMapping |
| Spring Data JPA + Hibernate | ORM-based database operations and CRUD repositories |
| Spring Security + JWT | Authentication, authorization, and JWT token management |
| Maven | Dependency management and project build lifecycle (pom.xml) |

## 7.3 Database & Tools

| Tool / Technology | Purpose |
|---|---|
| PostgreSQL | Primary RDBMS – persistent storage of all application data |
| pgAdmin | GUI client for PostgreSQL database management and query testing |
| Postman | REST API testing, endpoint verification, and auth token testing |
| VS Code / IntelliJ IDEA | Primary IDE for backend (Java) and frontend (React.js) |
| Git & GitHub | Version control, source code management, collaboration |

# 8. Project Screenshots

## 8.1 Project File Structure

The project is organized as a monorepo with two main folders: backend (inventory-backend — Spring Boot Java project) and frontend (inventory-frontend — React.js Vite project). The backend follows a layered package structure: config, controller, dto, model, repository, and service.

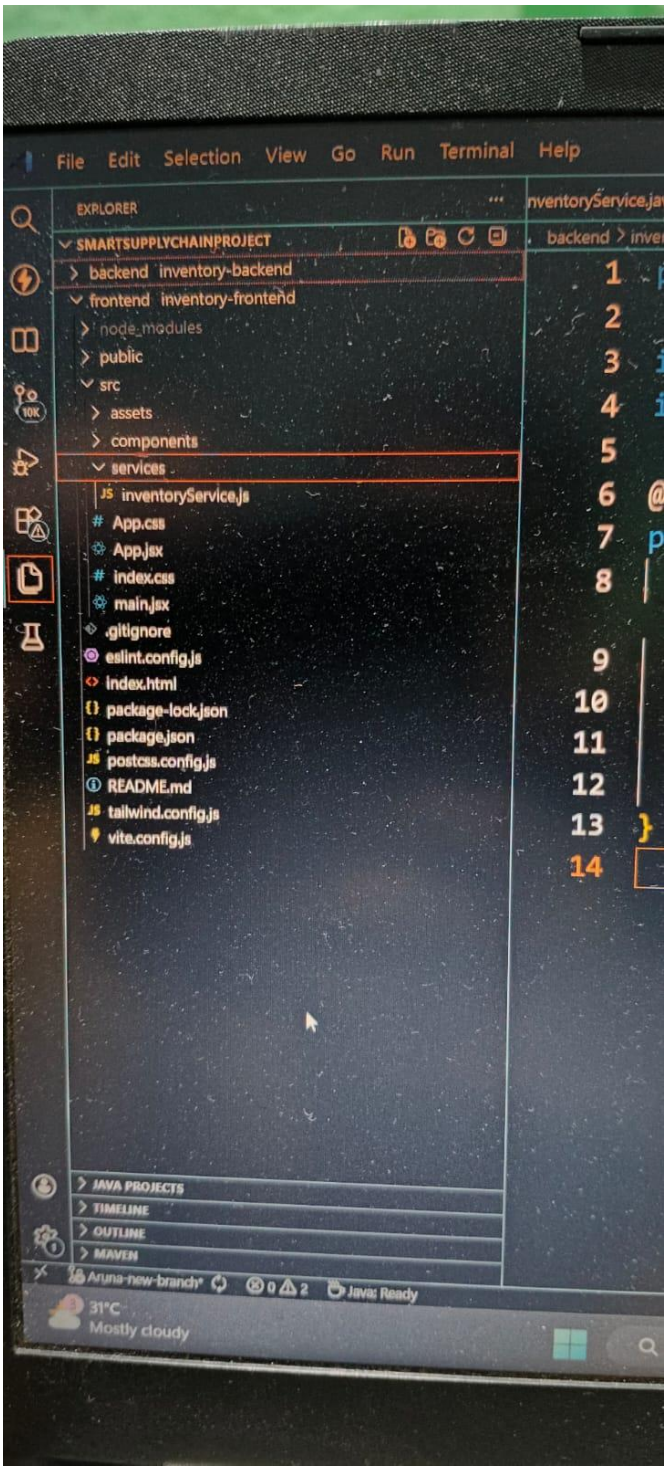


*Fig 3a: Frontend Project Structure (React.js / Vite)*

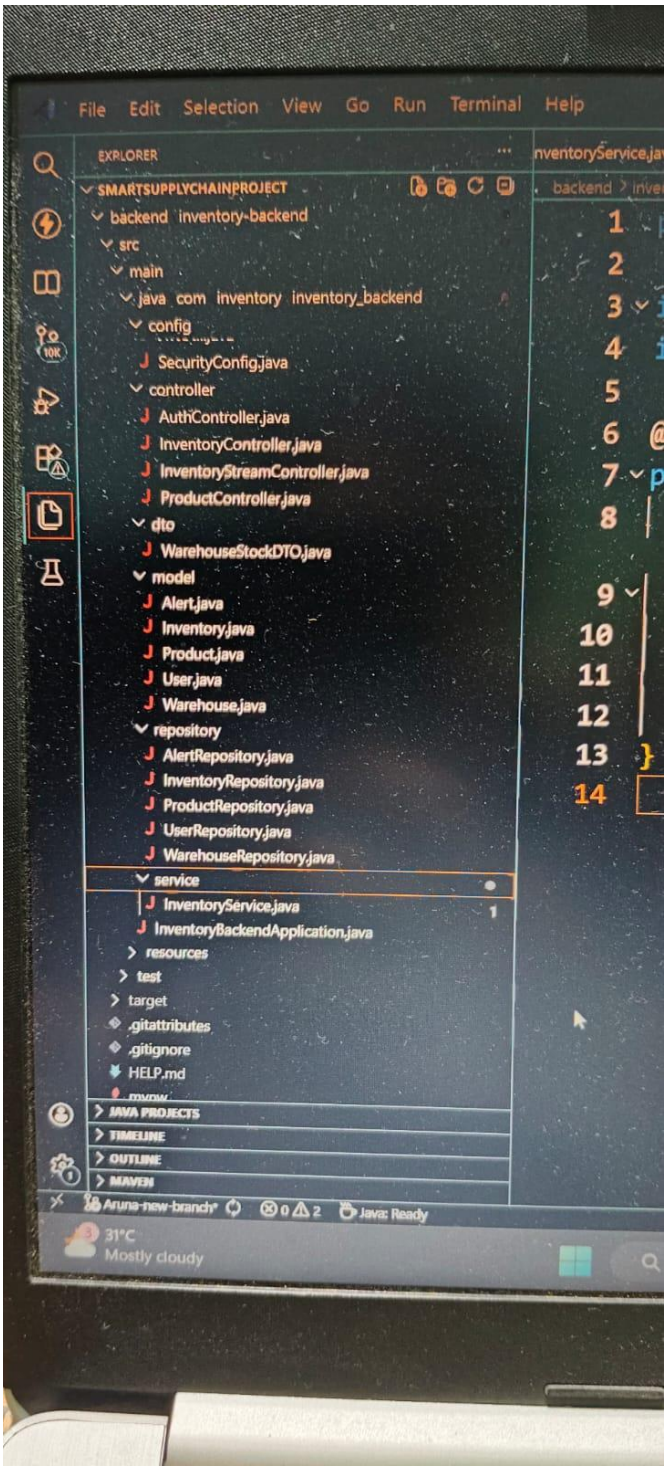


*Fig 3b: Backend Project Structure (Spring Boot / Java)*

## 8.2 Backend – Main Application Entry Point

InventoryBackendApplication.java is the Spring Boot entry point, annotated with @SpringBootApplication. It bootstraps the embedded Tomcat server and loads all auto-

configured beans including Spring Security and JPA.

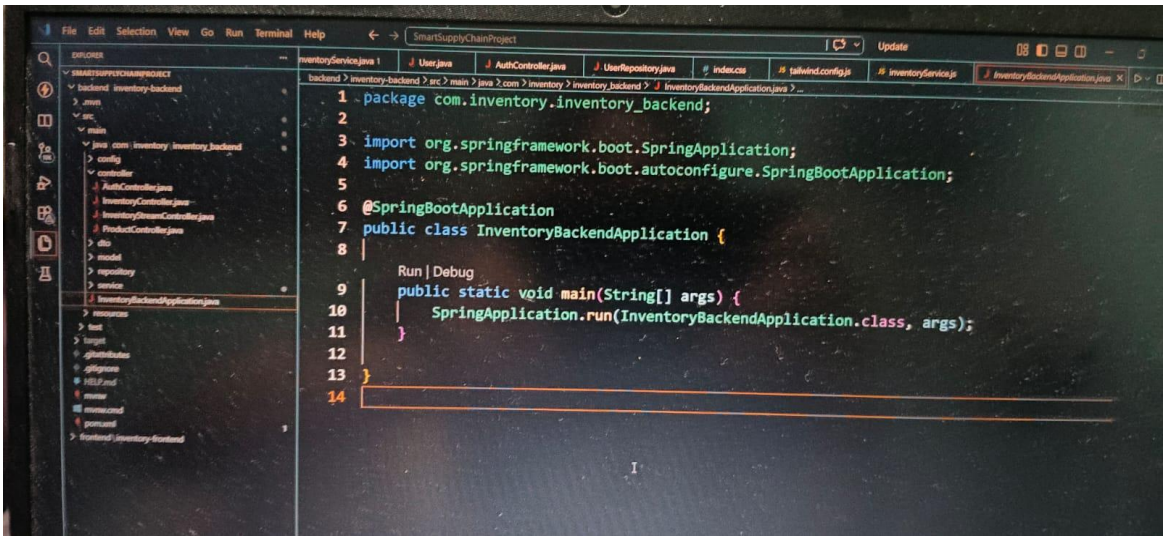


*Fig 4: InventoryBackendApplication.java – Spring Boot Entry Point*

## 8.3 Frontend – inventoryService.js (API Layer)

inventoryService.js defines all API communication functions. The base URLs for the backend endpoints (inventory, products, auth) are declared as constants. The loginUser function demonstrates a POST request to the auth endpoint returning a JWT token on success.

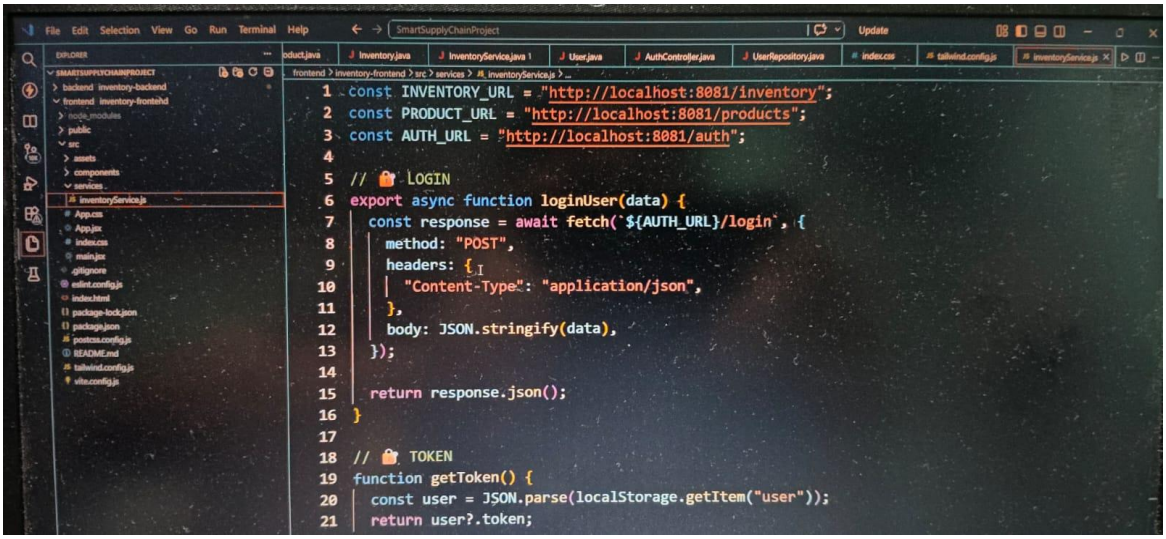


*Fig 5: inventoryService.js – Frontend API Service (Login & Token Management)*

## 8.4 Frontend – App.jsx (React Component Imports)

App.jsx imports all service functions (getAllInventory, addInventory, deleteInventory, updateQuantity, loginUser, getAllProducts) from inventoryService.js. It also registers Chart.js components (CategoryScale, LinearScale, BarElement, Tooltip, Legend) for the stock visualization bar chart using react-chartjs-2.

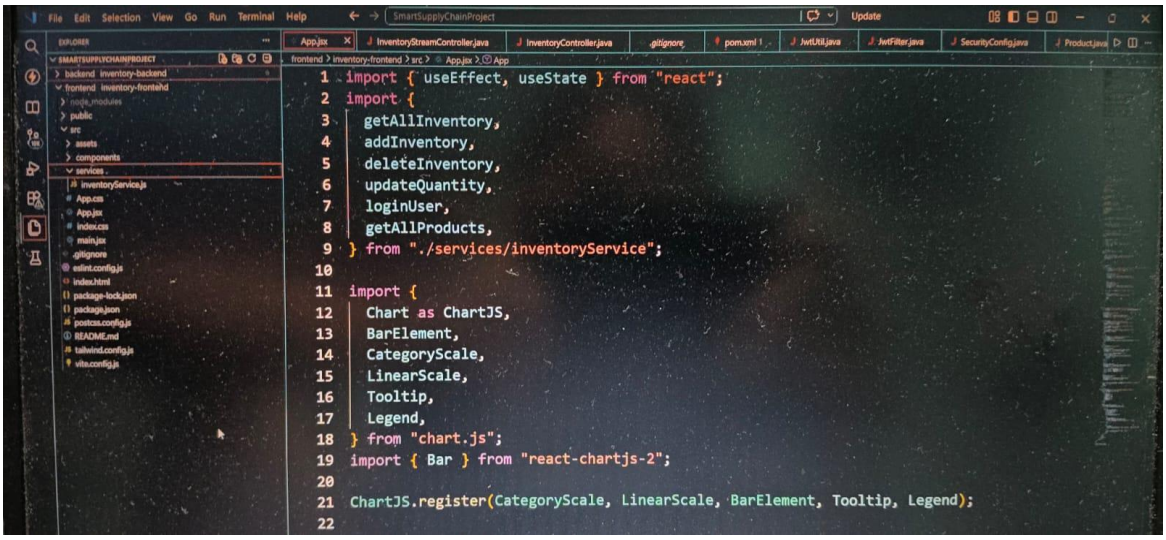


*Fig 6: App.jsx – React Imports including Chart.js Integration*

## 8.5 Database – Product Table (pgAdmin)

The product table in PostgreSQL stores 6 products across two categories — Electronics (Mobile, Tablet, Monitor, Laptop) and Accessories (Keyboard, Mouse). The table shows product_id, category, expiry_date, name, and other fields. Data was verified using the query: SELECT * FROM product;

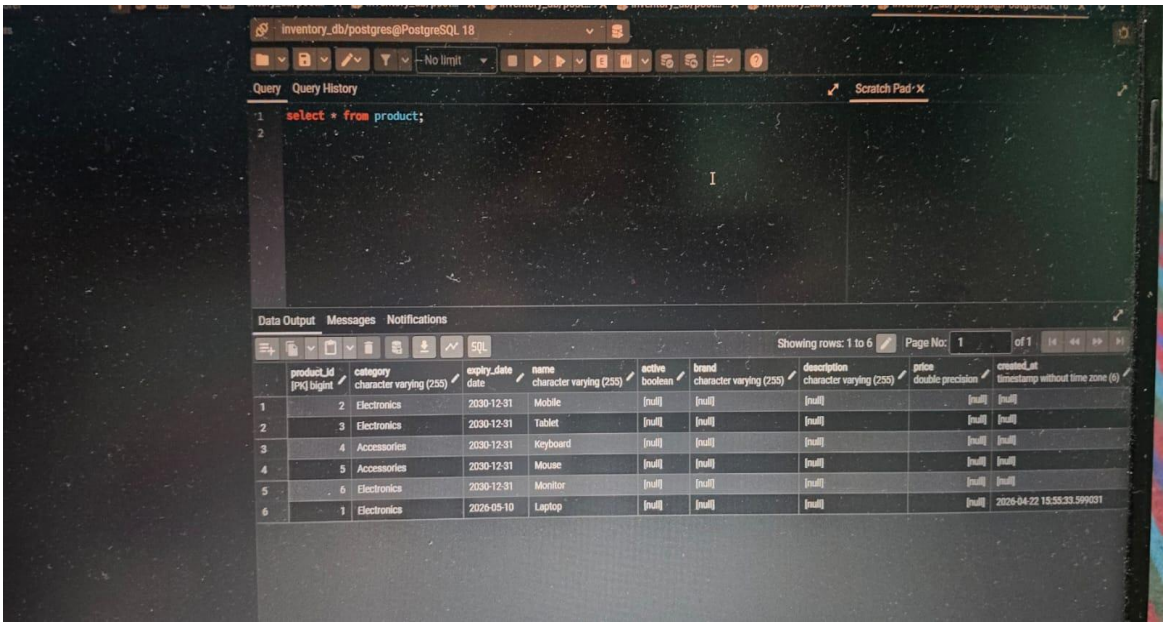


*Fig 7: PostgreSQL – Product Table Data (pgAdmin Query Output)*

## 8.6 Database – Inventory Table (pgAdmin)

The inventory table records stock quantities for each product-warehouse combination. The table shows inventory_id, last_updated timestamps, product_id (FK), quantity, warehouse_id (FK), expiry_date, and stock_status. Real inventory records are visible with timestamps from March–April 2026.

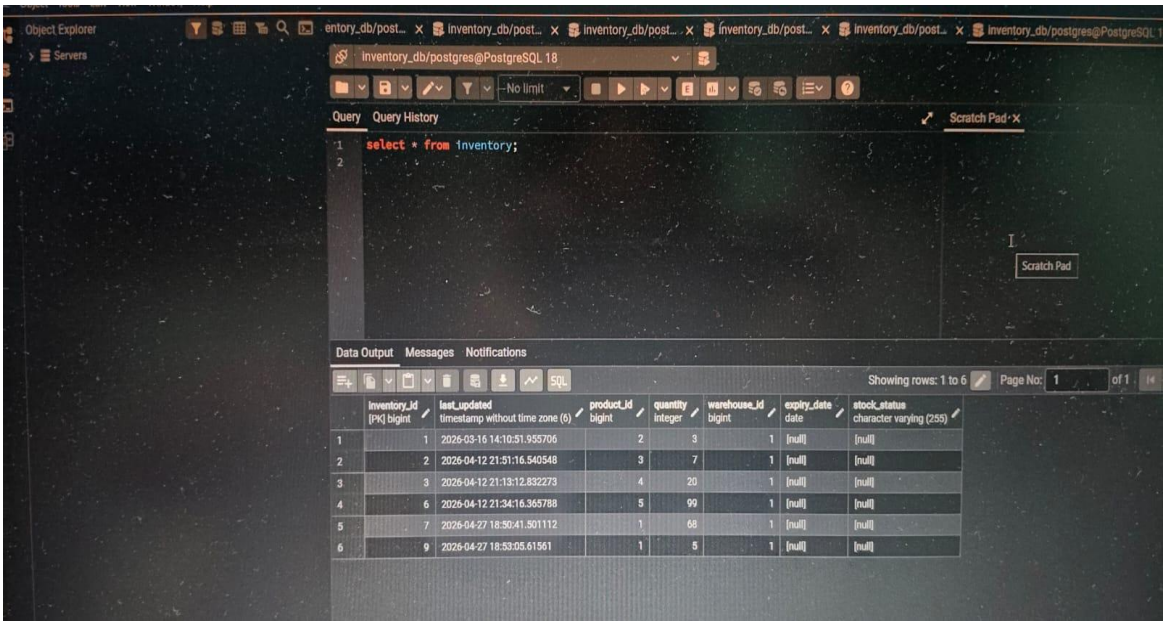


*Fig 8: PostgreSQL – Inventory Table Data (pgAdmin Query Output)*

## 8.7 API Testing – GET /products (Postman)

The GET /products endpoint returns a JSON array of all products with their full details: category, createdAt, expiryDate, maxStockLevel, minStockLevel, name, and productId. The API returned a 200 OK response in 25ms, confirming correct backend operation.

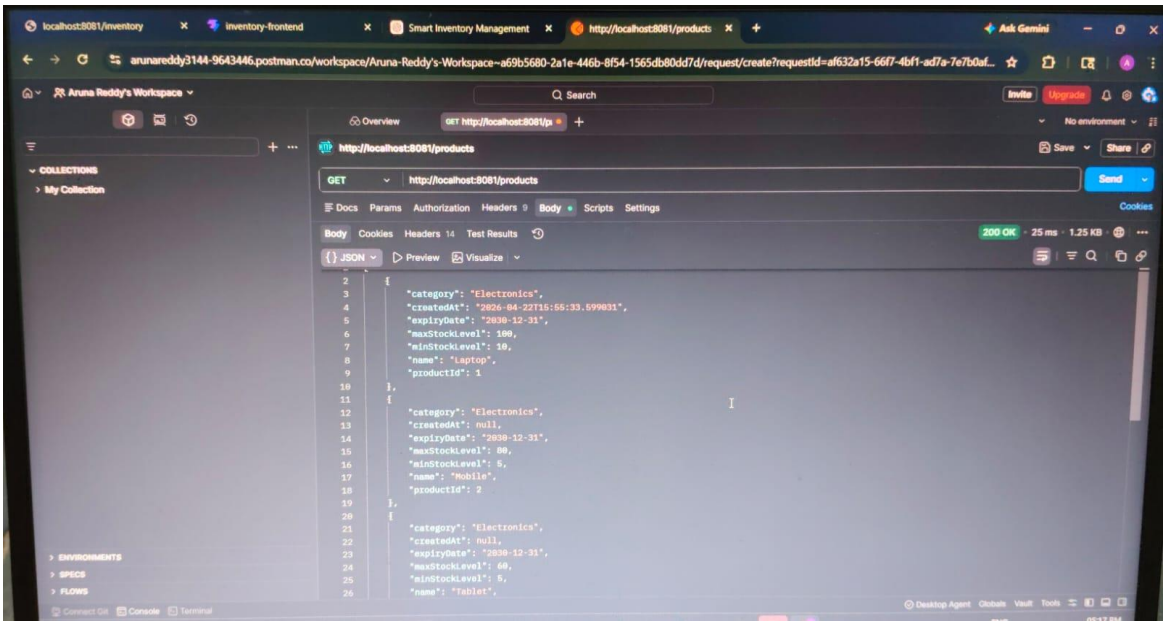


*Fig 9: Postman – GET http://localhost:8081/products → 200 OK Response*

## 8.8 API Testing – POST /auth/login (JWT Authentication)

The POST /auth/login endpoint accepts username and password in the request body and returns a JSON response with success: true, a login success message, the signed JWT token, and the username. The response was received in 30ms with 200 OK status, confirming the JWT authentication flow works correctly end-to-end.

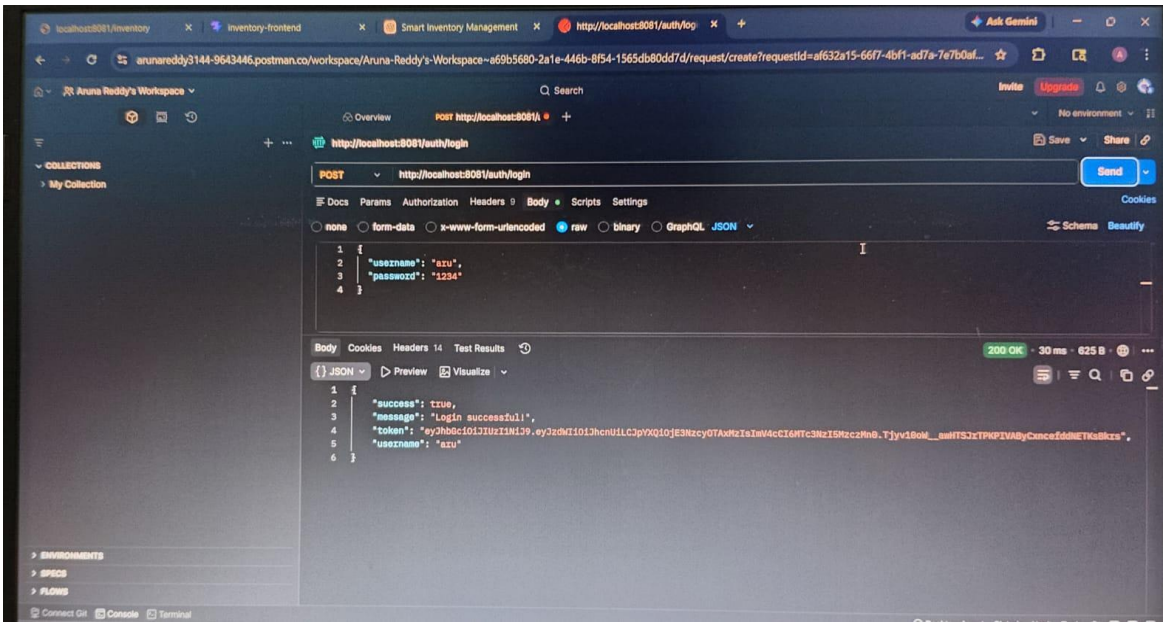


*Fig 10: Postman – POST /auth/login → 200 OK with JWT Token Response*

## 8.9 Live API Response – GET /inventory (Browser)

Accessing the inventory endpoint directly in the browser returns the full JSON payload with nested product and warehouse objects. Each inventory record includes inventoryId, lastUpdated, product object (with all product fields), quantity, and warehouse object (with location: Delhi, managerName: Rahul).

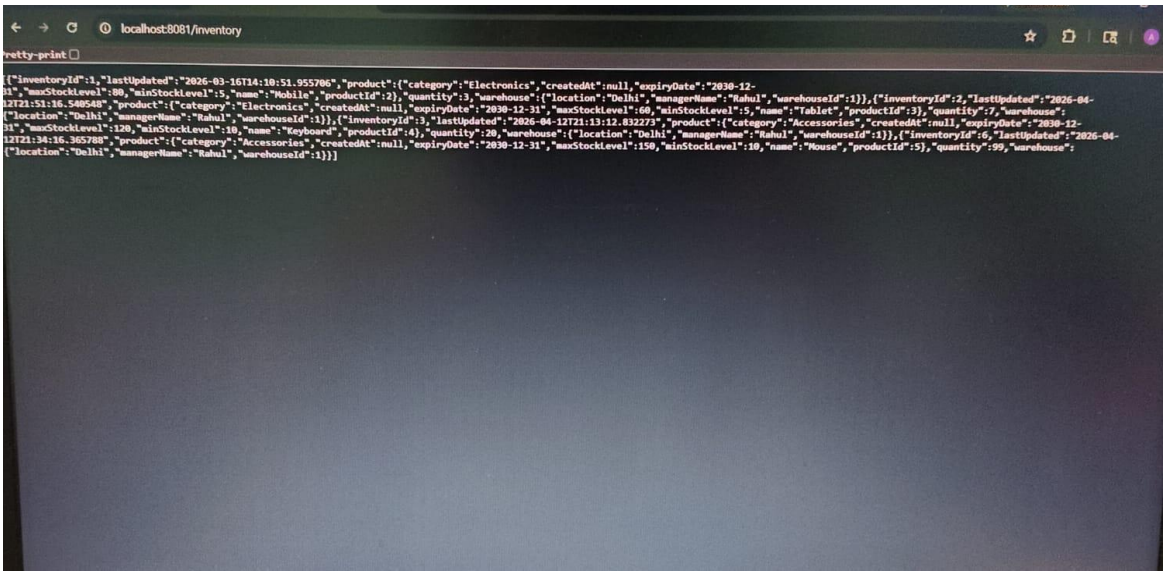

*Fig 11: Browser – GET localhost:8081/inventory JSON Response*

## 8.10 Inventory Dashboard – Stock Overview (Chart View)

The Inventory Dashboard displays three KPI cards: Total Records, Total Stock, and Low Stock (highlighted in red when below minimum). The Stock Overview bar chart (powered by Chart.js) visualizes stock quantities for all products — Mobile, Tablet, Keyboard, Mouse, Monitor, Laptop — at a glance. The search bar filters inventory records in real time.

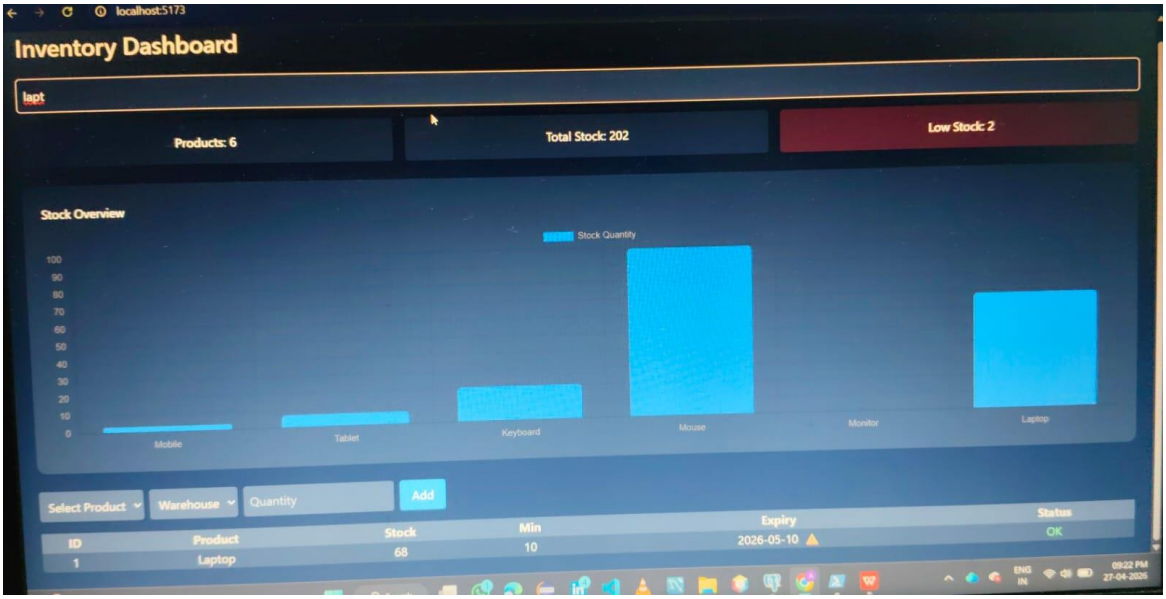


*Fig 12: Inventory Dashboard – KPI Cards and Stock Overview Bar Chart*

## 8.11 Inventory Dashboard – Inventory Table with Actions

The lower section of the Inventory Dashboard displays a detailed inventory table with columns: ID, Product, Warehouse, Quantity, and Actions. Each row includes Edit (yellow) and Delete (red) action buttons for real-time CRUD operations. The form at the top allows adding new inventory records by selecting Product ID, Warehouse ID, and entering Quantity.

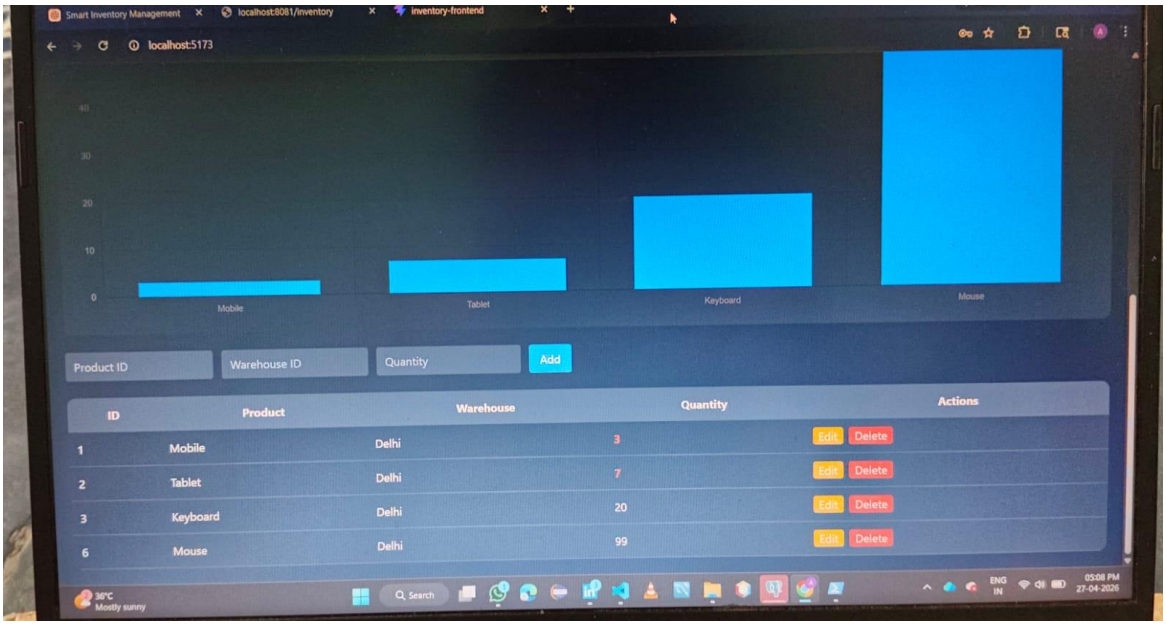


*Fig 13: Inventory Dashboard – Inventory Table with Edit/Delete Actions*

## 8.12 Dashboard – Early Version (Stock Overview)

This screenshot shows an earlier iteration of the Inventory Dashboard with the Stock Overview chart displaying 4 inventory records (Total Records: 4, Total Stock: 129, Low Stock: 2). The bar chart highlights how stock levels are dynamically rendered based on live data from the backend API, with the tallest bar representing Mouse (99 units).

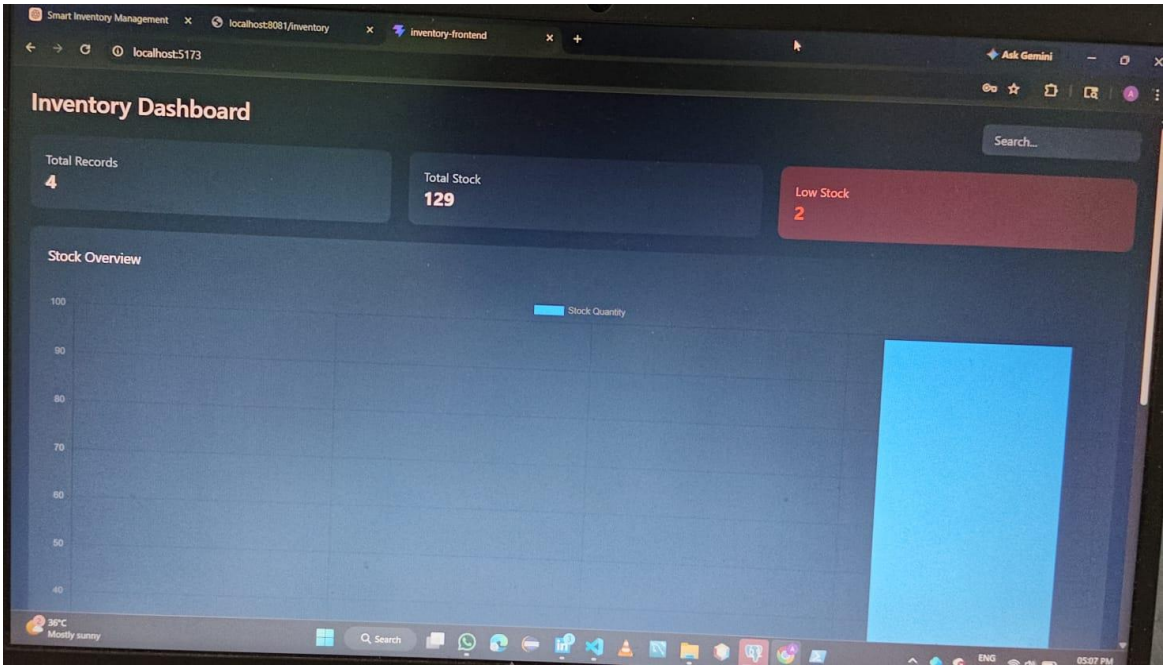


*Fig 14: Early Dashboard Version – Stock Overview Chart (4 Records)*

# Thank You